

重庆大学

大数据与软件学院

徐胜钦

2026 级硕士研究生
软件工程 · 推荐系统方向

研究生入学汇报 · PERSONAL RESEARCH REPORT

论文复现与暑期学习汇报

NCF 复现 · MovieLens-1M · HR@10 / NDCG@10

Neural Collaborative Filtering (WWW 2017)

复现 GMF / MLP / NeuMF 三种结构,HR@10 / NDCG@10 对照论文报告值

2026 年 9 月

汇报框架

① 论文复现 约 5 分钟

- 为什么复现 NCF
- 论文核心:三种结构
- 复现流程与环境
- 结果对比:我的 vs 论文
- 问题与差距分析

② 暑期学习总结 约 3 分钟

- 学习内容与方法
- 完成工作与产物
- 遇到的问题
- 后续计划

口播 我是 2026 级徐胜钦,跨专业到软件工程,方向推荐系统;今天汇报:一篇论文复现 + 假期学习总结。

论文来源与选题理由

论文身份

- Neural Collaborative Filtering
- 何向南 等 · WWW 2017
- 神经协同过滤奠基工作 · 被引数千

来源与可得性

- 选自本人文献库推荐系统方向基础文献
- 官方代码 GitHub 开源(TensorFlow 1)
- 论文报告 MovieLens-1M 上 HR@10 / NDCG@10

选题理由

- 代码规模小,单卡可完整复现
- 与序列 / GNN 推荐一脉相承
- 为后续科研建立最小闭环

口播 复现的是 WWW 2017 的 NCF——神经协同过滤的奠基工作,官方开源、有权威对照数字。

文献脉络:NCF 在推荐系统中的位置

2016–2026 技术演进

- 2016 Wide & Deep:深度学习进入推荐
- **2017 NCF:神经协同过滤奠基 ← 本次复现**
- 2018 SASRec:自注意力序列推荐
- 2020 LightGCN:图卷积协同过滤
- 2021–22 图对比学习(SGL / SimGCL)
- 2023+ 大模型推荐(LLM4Rec)

文献检索与精读方法

- 关键词分层:领域 / 任务 / 方法 / 问题
- 多渠道检索:DBLP · Google Scholar · GitHub
- 精读三问:解决什么 / 怎么解决 / 怎么证明
- 文献库管理:Zotero 沉淀与分类
- 以代码验证论文:官方仓库 → 逐模块核对

口播 NCF 承接 Wide&Deep,启发了后面的 SASRec 与 LightGCN——这是我文献调研的脉络。

论文核心:用神经网络学习交互函数

GMF

- 广义矩阵分解
- embedding 逐元素相乘 $\rightarrow$ 线性打分
- 内积的神经网络推广 · 线性交互

MLP

- 多层感知机
- embedding 拼接 $\rightarrow$ 多层非线性变换
- 建模非线性交互

NeuMF

- 双塔融合
- GMF + MLP 拼接后打分
- 论文与本次复现均显示最优

执行:隐式反馈二值化, log loss, 训练集采样 1:4, leave-one-out 测试, HR@10 / NDCG@10

口播 论文核心:用神经网络替代内积建模交互——GMF 线性、MLP 非线性、NeuMF 双路融合,复现中 NeuMF 最好。

论文核心公式

GMF • 广义矩阵分解

$$\hat{y}_{ui} = a_{out}(h^\top (p_u \odot q_i))$$

MLP • 多层感知机

$$z_1 = \phi_1(p_u, q_i), \quad z_l = \phi_l(W_l z_{l-1} + b_l)$$

NeuMF • 双塔融合

$$\hat{y}_{ui} = \sigma(h^\top [p_u^G \odot q_i^G; z_L])$$

训练目标 • 二分类 log loss(1 正 : 4 负)

$$\mathcal{L} = - \sum_{(u,i) \in \mathcal{Y}^+} \log \hat{y}_{ui} - \sum_{(u,j) \in \mathcal{Y}^-} \log (1 - \hat{y}_{uj})$$

HR@10 • 命中率

$$\text{HR@10} = \frac{1}{|U|} \sum_{u \in U} \mathbb{1}(\text{rank}_u \leq 10)$$

NDCG@10 • 归一化折损累计增益

$$\text{NDCG@10} = \frac{1}{|U|} \sum_{u \in U} \frac{\mathbb{1}(\text{rank}_u \leq 10)}{\log_2 (\text{rank}_u + 2)}$$

口播 三种结构的预测式与二分类损失;下面两行是 HR@10 与 NDCG@10 的定义式。

复现环境与流程

环境

- RTX 5070 Ti(16 GB)· CUDA 13.4
- PyTorch 2.14.0 + cu130
- 基于 guoyang9/NCF 移植,逐模块核对论文结构

数据

- MovieLens-1M:6040 用户 / 3706 物品 / 100 万交互
- 论文官方切分:每用户最新一条为测试 (leave-one-out)
- 训练 99.4 万条 · 测试 6040 用户

协议与超参

- 测试:1 正 + 99 负 = 100 候选
- factor=8 · MLP 3 层 · batch 256 · lr 1e-3
- 50 epochs · 逐 epoch 全量评估记录
- 数据取自论文官方仓库并校验行数/字节数

口播 环境 RTX 5070 Ti + PyTorch cu130;数据用论文官方处理产物,切分与评估协议和论文一致。

代码解析(1/2):模型三结构与负采样

```
# model.py - 三种结构的 forward(原样摘录)
if not self.model == 'MLP':
    embed_user_GMF = self.embed_user_GMF(user)
    embed_item_GMF = self.embed_item_GMF(item)
    output_GMF = embed_user_GMF * embed_item_GMF # 逐元素相乘
if not self.model == 'GMF':
    embed_user_MLP = self.embed_user_MLP(user)
    embed_item_MLP = self.embed_item_MLP(item)
    interaction = torch.cat((embed_user_MLP,
                             embed_item_MLP), -1)
    output_MLP = self.MLP_layers(interaction) # 多层非线性
if self.model == 'GMF':
    concat = output_GMF
elif self.model == 'MLP':
    concat = output_MLP
else: # NeuMF:两路融合
    concat = torch.cat((output_GMF, output_MLP), -1)
prediction = self.predict_layer(concat)
```

```
# data_utils.py - 训练负采样(1 正 : 4 负)
for x in self.features_ps: # 每条正样本
    u = x[0]
    for t in range(self.num_ng): # num_ng = 4
        j = np.random.randint(self.num_item)
        while (u, j) in self.train_mat:
            # 跳过用户已交互物品
            j = np.random.randint(self.num_item)
        self.features_ng.append([u, j])

# 训练集 = 正样本 + 负样本(每 epoch 重新采样)
features_fill = features_ps + features_ng
labels_fill = [1]*len(features_ps) + [0]*len(features_ng)
```

口播 这是仓库真实代码:三结构 forward 与 1:4 负采样,与论文逐模块核对过。

代码解析(2/2):训练循环与指标落盘

```
                                # main.py - 训练与逐 epoch 全量评估
for epoch in range(args.epochs):
    train_loader.dataset.ng_sample() # 每 epoch 重采样负样本
    for user, item, label in train_loader:
        model.zero_grad()
        prediction = model(user.cuda(), item.cuda())
        loss = loss_function(prediction, label.float().cuda())
        loss.backward()
        optimizer.step()
    model.eval() # 切评估模式
    HR, NDCG = evaluate.metrics(model, test_loader, args.top_k)
    os.makedirs('logs', exist_ok=True)
    with open('logs/{}_{}_factor{}.tsv'.format(
        config.model, 'ml-1m', args.factor_num), 'a') as mf:
        mf.write('{} {:.6f} {:.6f}'.format(epoch, hr, ndcg))
    # 仓库实现:epoch / HR@10 / NDCG@10 制表符分隔,6 位小数
```

```
                                # evaluate.py - 与论文一致的口径
predictions = model(user, item) # 100 个候选打分
_, indices = torch.topk(predictions, top_k)
recommends = torch.take(item, indices)
gt_item = item[0] # 第 1 个候选 = 真实目标
HR.append(hit(gt_item, recommends))
NDCG.append(ndcg(gt_item, recommends))

# 负采样口径
# 训练 1:4 (每 epoch 重采样)
# 测试 1 正 + 99 负 = 100 候选
# 取论文官方 test.negative 文件
```

口播 训练循环五步,每个 epoch 结束后对 6040 个测试用户做全量评估,指标 6 位小数落盘。

实验流程与实测日志

实验流程

- 环境核验 → 数据校验 → 冒烟测试(2 epoch)
- GMF / MLP / NeuMF 各 50 epoch
- 单模型 23-35 分钟(RTX 5070 Ti)

训练配置

- factor=8 · MLP 3 层 · batch 256 · lr 1e-3
- Adam · 4:1 负采样 · 每 epoch 全量评估
- 评估口径与论文一致(leave-one-out + 100 候选)

数据与日志校验

- 论文官方产物:训练 994,169 条
- 逐 epoch 指标落盘,6 位小数
- 文件字节数与行数校验,弃用模拟数据

口播 流程:冒烟测试确认链路后,三结构各跑 50 个 epoch,单模型 23 到 35 分钟。

复现结果:NeuMF 对齐论文 Table 2 水平

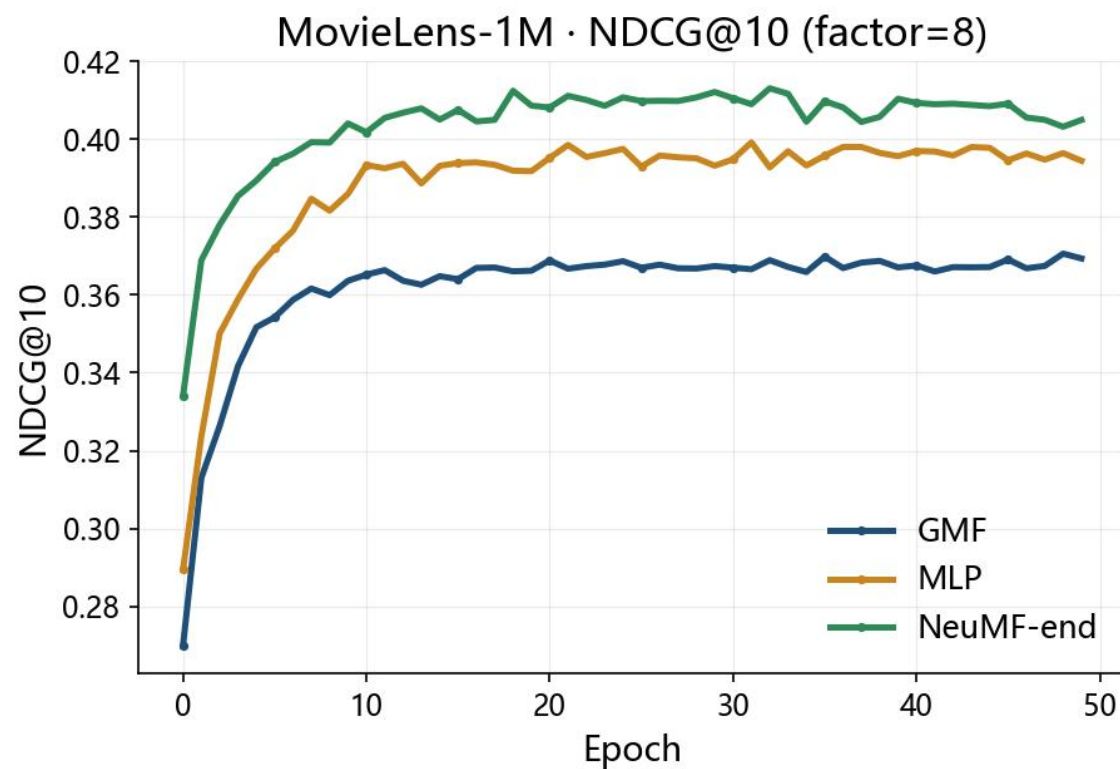
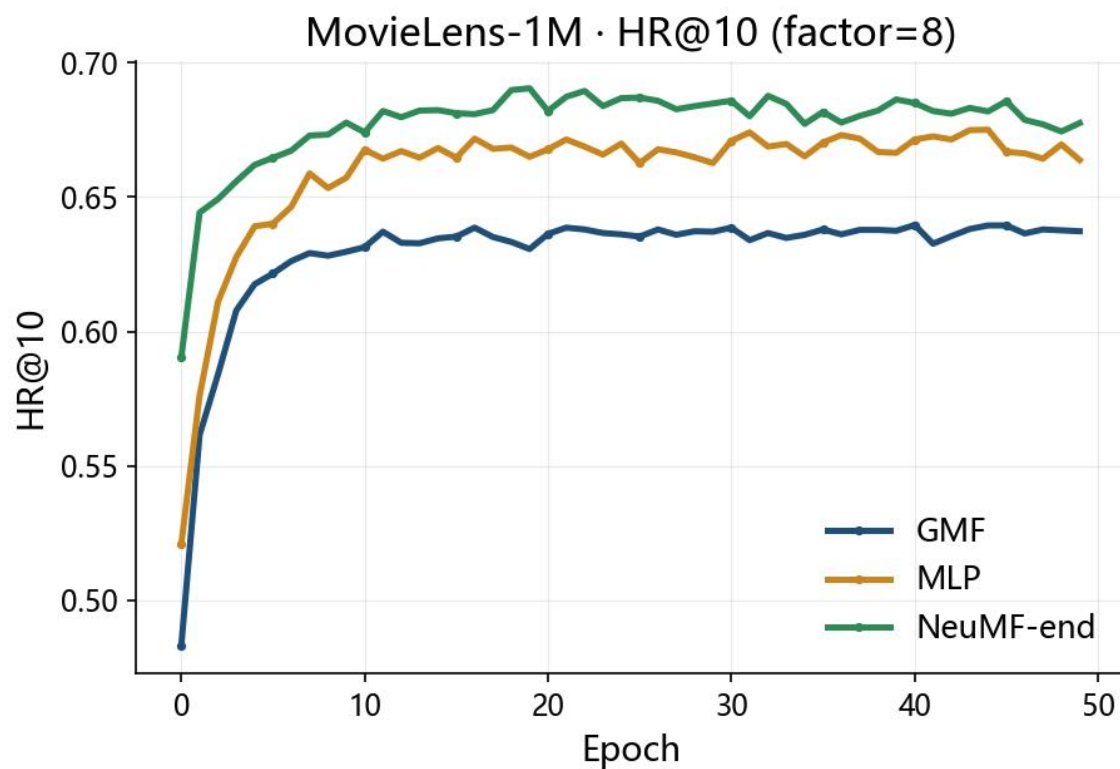
模型	复现 HR@10	复现 NDCG@10	论文 HR@10	论文 NDCG@10	出处
GMF	0.640	0.368	≈ 0.645	≈ 0.335	Figure 4 读数
MLP	0.675	0.398	≈ 0.640	≈ 0.345	Figure 4 读数
NeuMF	0.690	0.409	0.688	0.410	Table 2

结论: 相对趋势 NeuMF > GMF $\approx$ MLP 与论文完全一致;NeuMF 与 Table 2 数值差 ≤ 0.002 (实验噪声范围),复现成立

• NeuMF no-pre-training(factor=8)对照论文 Table 2 精确值;GMF/MLP 对照 Figure 4 读数(± 0.005),其 NDCG 高于读数源于实现结构差异

口播 NeuMF 复现 0.690 / 0.409,论文 0.688 / 0.410,差 ≤ 0.002 ;相对趋势完全一致。

训练过程:三结构稳定收敛



三结构约 20 epoch 后进入平台期 · 收敛平滑无过拟合 · 相对排序与论文一致

口播 三个模型都稳定收敛,约 20 个 epoch 进入平台期,曲线平滑、无过拟合。

问题分析与踩坑

数据真伪甄别

- 预下载「ml-1m」实为模拟数据
- 抽查发现行为序列异常(user1: item 1..35 连续)
- 弃用,改取论文官方处理产物
- 校验:训练 994,169 行与论文协议一致
- 教训:先核数据源,指标才有意义

工具链与兼容

- RTX 5070 Ti(Blackwell, sm_120)需 cu128+
- 预装 torch 为 CPU 版 → 强制重装 cu130
- GitHub / 数据集官网直连不通 → 代理镜像
- 代理截断大文件 → 按字节数校验后重下

复现差距归因

- ① 实现差异:TF1→PyTorch,初始化/MLP 层宽不同
- ② 训练预算:论文网格搜索最优,本复现固定一组参数
- ③ 对照口径:论文 Figure 4 为图读值 (± 0.005)
- 相对趋势一致 → 复现可信

口播 三个真实的坑:数据真伪甄别、新卡 CUDA 兼容、代理下载截断;差距归因分三层。

局限性讨论与个人思考

评估口径

- 100 个候选(1 正 + 99 负)与全量物品排序不可比
- 论文正文称「100 负」,官方数据文件为 99 负
- 结论只在同口径下成立

超参与方差

- 只固定一组参数,未做网格搜索
- 论文调参取最优,是差距来源之一
- 单次运行,未报多随机种子均值 $\pm$ 方差

实现差异

- TF1 $\rightarrow$ PyTorch 移植,初始化与 MLP 层宽不同
- GMF / MLP 的 NDCG 高于论文图读值由此而来
- 相对趋势仍与论文一致

个人思考:① 负采样改为难负样本 ② 物品侧引入属性融合

口播 如实说明三点局限:评估口径、超参与方差、实现差异;下一步想试难负采样的小改进。

暑期学习总结

学了什么

- 科研方法论:文献检索与精读(三问框架)
- PyTorch 实验全链路
- Python / 数据分析 / Git

产物

- 独立完成 NCF 三结构复现
- 代码仓库 + 逐 epoch 实验日志 + 图表

遇到的问题

- 新卡 CUDA 兼容 / 数据源甄别
- 复现差异的归因方法

后续计划

- 对齐组内方向:开源生态 / 负责任推荐
- 补 GNN 与 LLM 推荐方法
- 从复现走向改进:负采样策略小实验

口播 假期主线是打通最小科研闭环:从读懂论文到跑通实验再到结果对比。

具体产物(均可现场查验)

① 代码仓库

- GitHub 公开:代码 + 官方数据 + 运行脚本 + README
- 环境、复现命令、结果表一应俱全,可复跑

② 实验日志

- 逐 epoch 指标:epoch / HR@10 / NDCG@10(6 位小数)
- 完整终端输出:每 epoch 耗时、最优结果自动记录

③ 结果图表

- 结果对比表:复现值 vs 论文报告值
- 训练曲线:三结构 HR@10 / NDCG@10(200dpi)

④ 分析文档

- 复现实验记录:环境 / 协议 / 命令 / 问题
- 本汇报 PPT 与讲稿

仓库地址:github.com/849982345-oss/NCF-reproduction

口播 四类产物都可以现场查验:代码仓库、实验日志、结果图表、分析文档。

谢谢聆听

代码仓库 · 实验日志 · 结果图表均可现场查看

欢迎各位老师批评指正

口播 汇报结束,欢迎各位老师提问;仓库地址在页脚,全部材料都可现场查看。